

Decentralised delta and Sequential Pruning

Prajwal Dutta

26 mars 2026

Research Group: AMASS LAB
Semester: Spring, 2026

1 Introduction

The semester work focused on communication-aware topology management for underwater multi-agent systems in HoloOcean. In this setting, each robot must coordinate with nearby robots while operating in a communication environment that is low-bandwidth, lossy, delayed, and sometimes bursty. These constraints make many conventional graph-maintenance methods unrealistic because they assume either reliable links or $O(n)$ -scale communication.

The solution pursued in this workspace was to use a communication-feasible distributed pruning protocol based on scalar hop-distance propagation. Instead of exchanging full distance vectors, each robot broadcasts only its current hop distance to the root, which keeps message size constant and matches underwater acoustic communication constraints much better. After this baseline was established and tested inside the HoloOcean workflow, the work was extended into sequential pruning so that parent changes could be committed more conservatively and with reduced topological flapping.

2 Project Goals

The semester objectives recovered from the workspace were:

1. Build a communication-feasible pruning algorithm suitable for underwater multi-agent simulation.
2. Integrate the pruning workflow into HoloOcean experiments and generate repeatable evaluation scripts.
3. Evaluate robustness under multiple communication models, including IID loss, bursty loss, bandwidth caps, and distance-dependent impairment.

4. Improve parent selection and visualization so the tree dynamics are interpretable and stable.
5. Extend the baseline into sequential pruning with committed topology, grace periods, and delayed deletion.

3 Workspace Structure and Recovered Scope

The work in the repository is split across two main directories:

- `holocean_pruning_project/`: the structured HoloOcean pruning project, including the installable pruning package, experiment runners, sweep scripts, notes, figures, and result tables.
- `holocean_sequential_pruning/`: the later standalone sequential-pruning workspace, including the updated algorithm and deterministic verification scripts for grace-period and delete-candidate behavior.

This separation is important. The HoloOcean experiment runner imports the package in `holocean_pruning_project/pruning/`, which implements the communication-feasible δ -BFS pruning baseline. The standalone sequential-pruning directory contains the next-stage extension that adds committed-parent logic and edge-state management, but its own README explicitly describes HoloOcean integration as the next step rather than already completed.

4 Technical Work Completed

4.1 Communication-Feasible Pruning Baseline

The core baseline algorithm implemented in the pruning project is a distributed spanning-tree pruning method that uses only scalar hop-distance propagation. Each message contains only $(sender_id, \delta_{to_root}, seq)$ and therefore remains constant-size at 6 bytes. This is the key design decision that makes the method realistic for underwater communication.

The pruning baseline keeps the following properties:

- $O(1)$ message payload.
- asynchronous operation with per-node broadcasts and jitter,
- configurable packet loss and delay,
- bandwidth accounting for attempted versus delivered traffic,
- spanning-tree extraction using locally chosen parent pointers.

At the same time, communication-infeasible phases from earlier formulations were intentionally removed. In particular, connectivity-assurance and robustness-enhancement phases that would require $O(n)$ vectors or all-pairs information were excluded from the simulator-facing implementation.

4.2 HoloOcean Integration and Experiment Infrastructure

The project was organized into a reusable HoloOcean experiment package. The workspace includes:

- editable package setup through `pyproject.toml`,
- HoloOcean runner scripts in `holoocean_runs/`,
- plotting utilities for publication-style figures,
- sweep scripts for parameter studies,
- result reconstruction utilities,
- notes documenting each iteration and validation step.

Two HoloOcean-facing control scripts were also prepared:

- `keyboard_control.py` for manual multi-agent control and visualization,
- `autonomous_control.py` for scripted multi-agent motion with obstacle-aware spawning, goals, and trajectory tracking.

The main experimental driver became `holoocean_runs/run_pruning_epoch.py`, which accumulated most of the semester’s simulator-side features: channel models, rate caps, queueing, tiebreak policies, diagnostics, visualization, and logging.

4.3 Bursty Gilbert-Elliott Channel Modeling

One major extension was the addition of a Gilbert-Elliott burst-loss model to capture clustered communication failures. This model uses a two-state Markov chain per directed link:

- a GOOD state with low packet-drop probability,
- a BAD state with high packet-drop probability,
- transition probabilities defined per send attempt.

This is an important improvement over IID loss because underwater channels often fail in bursts rather than as independent events. The project also added diagnostics to compare expected and observed burst statistics, along with figures showing burst-length CCDFs and drop-event rasters.

The validation notes show the intended effect clearly: under IID loss, the maximum consecutive-drop run was only 2 in the reference diagnostic, while the GE model produced a maximum run of 13. That confirmed that the implementation was generating clustered failures rather than just increasing average loss.

4.4 Parent Selection, Hysteresis, and Visualization Improvements

The work also improved how parents are chosen during tree formation. An ID-based tie-break policy caused the tree to collapse into an unrealistic star centered around the lowest-ID attractive node. This was corrected by adding a distance-based tie-break mode, with optional hysteresis so that a new parent must remain better for multiple comparisons before a switch occurs.

This change mattered because it made the resulting topology physically meaningful. The notes document a before/after comparison in which the ID-biased version produced a star, whereas the distance-based version produced a multi-depth tree with a more natural relay structure.

Visualization and debugging were also strengthened through:

- live matplotlib updates during simulation,
- clearer termination logging,
- explicit distinction between building edges and committed tree edges in the plots.

These changes made the protocol behavior much easier to inspect and debug at scale.

4.5 Bandwidth Cap and Queueing Model

The next major step was adding a per-robot transmit-rate cap with FIFO queues. This enabled direct study of how little bandwidth the pruning protocol actually needs.

The queueing model introduced:

- per-robot transmit budgets in bits per second,
- FIFO message queues,
- overflow policies,
- queue delay metrics,
- sweep scripts to study convergence against bandwidth.

This part of the project is especially important because it connects the abstract communication-feasible claim to quantitative performance limits.

4.6 Distance-Dependent Impairment Model

The simulator was then extended again with a distance-dependent drop model:

$$p_{drop_{eff}}(d) = clip(p_0 + k \cdot (d/R_{comm})^\alpha, 0, p_{max})$$

This model captures the fact that links near the communication radius should be less reliable than close-range links. Weak and strong impairment profiles were added, and a three-condition simulation study was performed for $N = 20$ across five random seeds.

4.7 Sequential Pruning Extension

After the HoloOcean pruning baseline and channel studies were in place, the work advanced into a standalone sequential-pruning phase. This extension added state needed to distinguish the unstable candidate topology from a more conservative committed topology.

The sequential-pruning version introduced the following new per-node concepts:

- *committed_parent*,
- *backup_parent*,
- per-neighbor *edge_status*,
- *delete_counter*,
- *grace_counter*,
- *last_heard_time*,
- *last_candidate*.

The sequential policy then used these fields to implement:

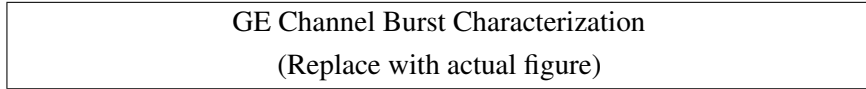
- committed-versus-candidate parent separation,
- backup-parent selection,
- grace-period retention when parent changes are detected,
- delayed deletion through a *DELETE_CANDIDATE* state,
- construction of the spanning tree from committed parents rather than directly from fast-changing candidates.

This was a meaningful algorithmic step beyond baseline δ -BFS because it targets a common practical problem: rapid parent switching under noisy or borderline communication conditions.

5 Experimental Results

5.1 GE Bursty-Loss Characterization

The workspace includes publication-ready channel figures in `holoocean_pruning_project/figures/`. The GE channel study demonstrated that bursty-loss behavior was correctly reproduced and measurably different from IID loss. The strongest GE profile showed heavy-tailed burst lengths and long consecutive-drop runs, which is precisely the kind of impairment the pruning protocol must tolerate underwater.



Figur 1: GE channel burst characterization

5.2 Rate-Cap Sweep: Main Quantitative Result

The strongest semester result is the rate-cap sweep for $N = 10$ and $N = 20$. These experiments measured whether the protocol could still form a valid spanning tree when each robot was assigned a strict transmit budget.

The recovered summary table from the workspace is:

Tabell 1: Rate-cap sweep results

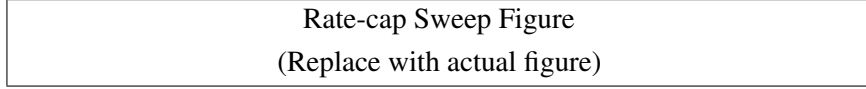
N	Cap (bps)	Description	Runs	Success	t_{tree} (s)	Delay p95 (ms)	rx (B/s)
10	inf	No limit	10	1.00	2.9 +/- 0.2	0	185
10	250	High cap	10	1.00	4.3 +/- 0.1	2135 +/- 359	169
10	48	2x above breakpoint	3	1.00	11.5 +/- 0.7	12483 +/- 1336	38
10	30	Above breakpoint	3	1.00	18.0 +/- 2.0	15094 +/- 2288	18
10	18	Near breakpoint (partial)	3	0.67	27.8 +/- 1.4	20700 +/- 317	10
10	12	Below breakpoint (failure)	3	0.00	-	22939 +/- 664	3
20	inf	No limit	3	1.00	3.0 +/- 0.1	0	546
20	250	High cap	3	1.00	5.0 +/- 0.1	4717 +/- 274	405
20	48	2x above breakpoint	3	1.00	15.0 +/- 0.5	15261 +/- 1891	67
20	30	Above breakpoint	3	1.00	20.6 +/- 0.8	19439 +/- 679	38
20	24	Near breakpoint (first success)	3	1.00	27.2 +/- 0.9	20317 +/- 584	27
20	18	Below breakpoint (failure)	3	0.00	-	20933 +/- 591	13

These results support several conclusions:

1. The protocol is highly bandwidth-efficient.
2. Convergence remains possible at extremely small per-robot rates.
3. The breakpoint scales only weakly with swarm size.

4. Delay rises dramatically near the breakpoint, but correctness is retained until the cap becomes too small.

For $N = 10$, the critical region was approximately 12 – 18 bps, with a marginal zone at 18 – 24 bps. For $N = 20$, the breakpoint shifted to approximately 18 – 24 bps and appeared as a sharper transition. The practical interpretation is that the protocol is constrained more by each robot’s ability to get enough fresh parent-related updates than by raw total network traffic.



Figur 2: Rate-cap sweep figure

5.3 Distance-Dependent Impairment Study

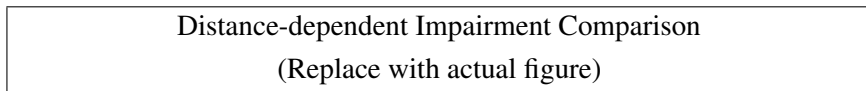
The distance-dependent impairment study evaluated whether tree formation remains reliable when long links are more likely to fail. Three conditions were tested for $N = 20$: baseline, weak impairment, and strong impairment. Each condition used five seeds.

The recovered aggregate results are:

Tabell 2: Distance-dependent impairment study results

Condition	Success	t_{tree_first} (s)	t_{tree_table} (s)	rx_Bps	Observed drop rate	Mean effective drop
Baseline	5/5	3.10 +/- 0.22	17.68 +/- 4.79	539	9.9%	10.0%
Weak	5/5	3.74 +/- 0.49	20.74 +/- 5.74	444	24.5%	24.97%
Strong	5/5	7.64 +/- 1.20	22.03 +/- 1.76	212	62.9%	62.37%

This was a strong robustness result. Even under the strong profile, where effective loss exceeded 60%, the protocol still converged in every trial. The main cost was slower initial tree formation and lower received throughput. In other words, the channel impairment reduced efficiency, but it did not break correctness over the tested horizon.



Figur 3: Distance-dependent impairment comparison

5.4 Sequential-Pruning Verification

The sequential-pruning work was verified through lightweight deterministic scripts in the standalone workspace. These scripts confirmed that:

- phase-2 state extension was added correctly,

- grace-period logic keeps the old committed parent during the grace window,
- delayed deletion keeps an edge in *DELETE_CANDIDATE* before fully removing it,
- committed-parent edges are used consistently in the maintained topology.

During this review, the following verification scripts were executed successfully:

- `holocean sequential pruning/verify_phase2.py`
- `holocean sequential pruning/verify_grace_deterministic.py`
- `holocean sequential pruning/verify_delete_candidate_deterministic.py`

This means the sequential-pruning mechanics are not only described in the README but also exercised by explicit deterministic checks.

6 Key Contributions of the Semester

The work completed over the semester can be summarized as the following concrete contributions:

1. Built a communication-feasible δ -BFS pruning baseline with constant-size messages suitable for underwater communication constraints.
2. Integrated that pruning workflow into a reusable HoloOcean experiment package with runners, sweeps, plots, and result summaries.
3. Added a Gilbert-Elliott burst-loss model and validated its clustering behavior.
4. Fixed parent-selection artifacts through distance-based tie-breaking and hysteresis.
5. Added live visualization and improved termination/debug instrumentation.
6. Implemented per-robot transmit-rate caps and FIFO queues to quantify the bandwidth breakpoint for successful tree formation.
7. Added a distance-dependent impairment model and showed 100% convergence in all tested $N = 20$ trials, even under strong degradation.
8. Developed a standalone sequential-pruning extension with committed parents, backup parents, grace periods, and delayed edge deletion.
9. Added deterministic verification scripts for the most important sequential-pruning state transitions.

7 Challenges and Lessons Learned

Several engineering lessons are documented directly in the workspace notes:

- simple tie-break rules can create unrealistic tree topologies,
- early stopping can bias channel-model statistics,
- per-link statistics become sparse quickly in short runs,
- live plotting in Python scripts requires explicit GUI event-loop handling,
- result sweeps need validation against raw JSON outputs to avoid silent data loss,
- queue-delay metrics and bandwidth limits must be interpreted per robot, not just globally.

More broadly, the semester work showed that communication realism is not a minor detail. Loss models, queueing, and rate caps directly change how fast the tree forms and whether it forms at all. This justified the shift from a purely algorithmic baseline toward a simulator-backed study with explicit channel models.

8 Limitations

The workspace also makes the current boundaries of the project clear.

1. First, the HoloOcean experiment pipeline is centered on the baseline pruning package, while the sequential-pruning logic currently lives in a separate standalone directory. That means the committed-topology logic has been implemented and verified, but not yet fully merged into the main HoloOcean runner used for the large sweeps.
2. Second, the sequential-pruning validation is currently strongest at the unit and deterministic-transition level. Large-scale comparative sweeps for the sequential version are still a natural next step.
3. Third, some of the notes mention tooling issues such as plotting-related non-zero exit codes during sweep automation. These were mitigated through CSV reconstruction and plotting fixes, but they still illustrate the need for careful post-run auditing.

9 Next Steps

The most logical next steps after this semester are:

1. integrate the sequential-pruning implementation into `holocean_pruning_project/holocean_runs/run`
2. rerun the main HoloOcean sweeps with committed-parent logic enabled,

3. compare baseline δ -BFS and sequential pruning on parent flapping, convergence time, and maintained-edge stability,
4. study parameter sensitivity for grace period G , delete threshold H , and backup-parent policies,
5. connect the pruned/committed topology directly to downstream CLF-CBF multi-robot control experiments.

10 Conclusion

The semester produced a solid and technically coherent body of work. The baseline pruning project demonstrated that a communication-feasible δ -BFS spanning-tree protocol can be implemented, instrumented, and evaluated in HoloOcean under realistic communication constraints. The experimental results showed that the method is robust to bursty loss, strong distance-dependent impairment, and extremely low per-robot bandwidth. On top of that, the later sequential-pruning extension addressed an important practical weakness of the baseline by introducing a committed topology with hysteresis-like behavior through grace periods and delayed deletion.

Taken together, the work establishes both a validated baseline and a credible next-stage extension. The baseline answers the question of whether pruning can be made communication-feasible in HoloOcean. The sequential extension answers how that baseline can be made more stable and deployment-oriented. This makes the semester’s work a strong foundation for the next phase of integrated multi-agent underwater coordination research.